

Lab 3: Classical Reinforcement Learning

1 Introduction

In this lab we will attempt to construct an agent capable of managing the race simulator from the previous lab. So far, we have tackled building agents in a stationary, contextual bandit setting in which actions in the current turn affect the state in the next turn only through the information the agent has to learn from. The race environment is not stationary; actions in the current turn affect the location on the track and the velocities available to the agent in the next turn. The reinforcement learning algorithms we've encountered that can be used to build an agent capable of dealing with a non-stationary environment are Monte Carlo algorithms and the SARSA TD(0) algorithm; we will use these to train our agent here, including an explicit strategy for ensuring continual exploration of the possible actions.

2 Set up the racetrack environment

Begin by slightly modifying the racetrack environment you developed for Lab 2. In particular, change the environment so that if the car crashes it is sent back to the start line and the episode continues. You may work with the code you developed or my version of the racetrack that I will provide. The full description of the modified environment follows.

The modified racetrack. Consider driving a race car around a turn. You want to go as fast as possible, but not so fast as to run off the track. The racetrack is a grid – it begins

with a vertical section 10 cells wide and 30 cells high, followed by a right turn into a horizontal section 15 cells wide and 10 cells tall. The starting line is the row of cells at the bottom of the first section. The finish line is the column of cells at the right of the horizontal section. The car begins at any cell on the starting line and each turn it is at one of the grid positions. The velocity is also discrete, a number of grid cells moved horizontally and vertically per time step. The actions are increments to the velocity components. Each may be changed by $+1$, -1, or 0 in each step, for a total of nine (3×3) actions. The vertical velocity component is restricted to be nonnegative. Both velocity components are restricted to be less than 5, and they cannot both be zero except at the starting line. Each episode begins in one of the randomly selected start states with both velocity components zero and ends when the car crosses the finish line. If the car attempts to go through the track boundary (anywhere but the finish line) it is automatically returned to a randomly selected position on the starting line, its velocity is set to 0, and the episode continues.

3 Learn about the environment

1. Set up the environment class by specifying the appropriate dictionary. In my version of the racetrack the config has the following structure:

```
height = 30    #set track height
width = 25    #set track width
v_track_width = 10    #set road width for the vert section
h_track_width = 10    #set road width for the horiz section
config = {'track_shape':{'height':height
                         ,'width':width
                         ,'v_track_width':v_track_width
                         ,'h_track_width':h_track_width}
          ,'starting_position':None
         ,'starting_velocity':None}    #set up the config
```

2. Experiment with this modified racetrack by running the step function. Simulate 100 races with randomly generated values for a (this will be different from the last lab because the environment has been modified).

4 Solving Markov Decision Problems via Monte Carlo & SARSA

1. Decide how you will represent the state information. Remember – this will be the only information that the agent has available to make decisions.

2. Decide how you will reward the agent. The goal of the agent is to reach to finish line without crashing and you need a reward function, $r(s', a, s)$, to incentivize the agent to do this. Begin by using the following reward function (provided in my version of

the racetrack environment):

$$r(s', a, s) = \begin{cases} \dfrac{1}{\max\left\{1, (x\text{–track width})^2 + \left(\dfrac{\text{horizonal track width - y}}{2}\right)^2\right\}} & \text{if } x, y \text{ not on finish line} \\ 10 & \text{otherwise.} \end{cases}$$

In addition, formulate a second reward function of your own design. We will test both below.

3. Implement an agent capable of taking actions in this environment by writing python code to execute the following algorithms.

   - On-policy, first visit Monte Carlo Control with ε-soft policies;
   - SARSA with ε-greedy policies;
   - Expected SARSA with ε-greedy policies.

   Train the agent by running these algorithms for an "appropriate" number of episodes (likely no more than 10,000).

4. Record episode length, average return, and success rate over training for each algorithm.

5 Wreck the Learning of your Agent!

State and reward information, environmental dynamics, hyperparameters, and algorithm choice are all super important in determining the learning dynamics of your agent. Your next task is to investigate how (some) of these things impact performance.

1. Construct at least two situations in which you expect these algorithms to perform poorly. Possible causes include poor reward design, incomplete state representations, poor exploration strategies, inappropriate hyperparameters, environmental changes, etc.

2. Prove that your changes degrade agent performance by rerunning your experiment from section 4 using your modified situation.

3. Choose one failure case and repair the learning of the agent (without simply undoing the change you made!).

Lab Assignment Report

Write up your analysis in a separate report of no more than 2 pages. Your report should answer the following questions:

1. Describe how you rewarded the agent (4.1) and how you conceptualized the state (4.2).

2. In your simulations in 3.2. how many turns does it take on average for the agent to cross the finish line with random actions? How often did the agent cross the finish line with the unmodified racetrack? How will the modification we made to the racetrack interact with the reward function to enhance or impede the agent's ability to learn?

3. Consider the process of training your agent in 4.3. For each of the algorithms and each of the reward functions (mine and yours) how many episodes did you train your agent for? How many turns does each episode take as training progresses? Why do you think there might be a relationship between training time and episode length? Provide visualizations that show the relationship between episode and episode length, average return, and success rate over training. Which algorithm is more sample efficient?

4. Choose two agents (two reward function/algorithm combinations). Compare the learned policy of these two agents. Specifically, create and provide a visualization of what each agent chooses to do at each location of the track.

5. How do the initial Q-values compare to the learned Q-values for the two algorithms?

6. What effect (if any) do γ and ε have on the performance of the racetrack agent?

7. Describe how you modified the training process to damage the learning of your agent. For each failure case explain what changed, how learning was impacted, and hypothesize how your changes impacted learning. Which algorithm was more robust, and what evidence supports your conclusions?

6 Submission Instructions

- Write up the answers to the questions in a short word document; aim for no more than 2 pages of written text and include all graphics generated. Add footnotes identifying which sentence addresses which questions. Write in complete sentences organized into paragraphs – your goal is to explain what you've done and what you've learned to your audience (me!). Include the appropriate plots you've generated as mentioned above. Convert this to pdf and submit it. Submit your .ipynb file and an .html of it as well.

- Please use the following naming conventions:
  - Lab_3_[YOUR NAME].pdf
  - Lab_3_[YOUR NAME].html
  - Lab_3_[YOUR NAME].ipynb

- The grading rubric for this assignment will be available in Canvas.

- NO OTHER SUBMISSION TYPES WILL BE ACCEPTED.

- You are welcome to use AI tools for implementation as you code up your solution, but the emphasis is on experimental design, analysis, and scientific reasoning. If you do this you must include all prompts used as an appendix in your written report or

provide a shareable link to them. Any uncited use of generative AI (i.e. prompts not provided) will be considered plagiarism. You may not use generative AI to write your report.